

oneM2M for Real-Time UxV Operations: A Multi-Protocol Performance Analysis

João Parreira ¹ , Pedro Barbeiro ² 

¹ Polytechnic University of Leiria; joao.s.parreira@ipleiria.pt

² Polytechnic University of Leiria; pedro.g.barbeiro@ipleiria.pt

Abstract

Unmanned vehicle operations require continuous telemetry delivery and low-latency command execution, increasingly at multi-vehicle scale. The oneM2M standard defines a service layer with native support for four transport bindings — WebSocket, MQTT, HTTP, and CoAP — but their comparative performance under real UxV workloads remains unknown. This paper presents the first empirical benchmark of all four oneM2M transport bindings on a live UxV platform, evaluating latency, throughput, packet loss, protocol overhead, and jitter across three operational scenarios and 120 runs. The central finding is that oneM2M is architecturally well-suited to UxV operations, but the reference implementation evaluated — ACME CSE v2025.11 with TinyDB — caps notification delivery at approximately 4–5 msg/s, rendering it not viable for multi-vehicle deployments where aggregate telemetry rates routinely exceed this ceiling. This bottleneck is a CSE implementation constraint, not a limitation of the oneM2M standard. Within the tested constraints, MQTT emerges as the most balanced binding, combining zero packet loss, the lowest telemetry jitter (15.7 ms), and competitive command latency (102.6 ms). The benchmark system is fully open-source and reproducible.

Keywords: oneM2M; unmanned vehicle; IoT middleware; protocol benchmarking; MQTT; WebSocket; CoAP; HTTP; telemetry; latency

1. Introduction

Unmanned vehicles (UxVs), encompassing aerial, ground, and maritime platforms, are increasingly used in demanding operations such as infrastructure inspection, search and rescue, and precision agriculture [1–4]. These applications require continuous telemetry delivery and low-latency command execution. As UxV deployments scale to multi-vehicle environments, point-to-point communication becomes unmanageable, and a standardized service layer that provides resource management, access control, and protocol interoperability becomes necessary [5].

The oneM2M standard [6] defines a service layer for Internet of Things (IoT) systems with properties relevant to UxV operations: a uniform resource model, subscription-based telemetry delivery, and native support for four transport bindings (Hypertext Transfer Protocol (HTTP) [7], WebSocket [8], Message Queuing Telemetry Transport (MQTT) [9], and Constrained Application Protocol (CoAP) [10]). This makes oneM2M-compliant middleware a strong candidate for UxV systems, where the ground station and the vehicle may have different connectivity constraints.

However, architectural suitability does not guarantee operational viability. Each transport binding introduces different latency and overhead characteristics, and it is not

Received:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Sensors* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

known whether all bindings meet the real-time requirements of UxV operations. Prior work has benchmarked IoT communication protocols in direct device-to-server configurations, evaluating metrics such as latency, jitter, and throughput, without the overhead of a oneM2M service layer [11,12]. Existing oneM2M studies either evaluate a subset of protocol bindings or do not evaluate performance on real hardware [13–15]. To the best of the authors' knowledge, no prior work has empirically compared all four oneM2M transport bindings on a live UxV platform.

This paper addresses a direct question: *is oneM2M a viable service layer for real-time UxV operations, and which transport binding best supports telemetry and command workloads?* A DJI Mavic 2 Enterprise Advanced drone communicates with an ACME Common Service Entity (CSE) instance over each of the four bindings across three operational scenarios: continuous telemetry uplink at 4 and 16 messages per second, and a command round-trip scenario in which 60 ping commands are dispatched and acknowledged by the UxV. Five metrics are collected: latency, throughput, packet loss, protocol overhead, and jitter. The benchmark architecture, integrating a commercial UxV platform with a oneM2M CSE, is fully reproducible and openly released.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes the system architecture. Section 4 defines the experimental methodology. Section 5 presents results. Section 6 discusses findings and limitations. Section 7 concludes.

2. Related Work

Several studies have compared IoT messaging protocols in direct device-to-server configurations. Wyrębowicz et al. [16] analyze MQTT, CoAP, HTTP, WebSocket, and Advanced Message Queuing Protocol (AMQP) based on their protocol specifications, comparing features such as transport type, Quality of Service (QoS) levels, and suitability for constrained devices, but without running experiments on real hardware. Silva et al. [11] go further by running measurements on a physical testbed, evaluating MQTT, CoAP, and OPC UA and finding that CoAP consistently achieves the lowest message delivery time. Amirkhanov et al. [12] connect an ESP32 microcontroller to a server and measure latency and connection stability for MQTT over Transmission Control Protocol (TCP), MQTT over WebSocket, and HTTP in a digital twin scenario. In all three cases, the device communicates directly with the server and there is no intermediate service layer, so the results do not capture the additional overhead that a standardized IoT platform such as oneM2M introduces.

Work specifically targeting oneM2M performance is limited in scope. Et-Tousy and Zyane [13] demonstrate how to configure MQTT and CoAP to work with a oneM2M platform, but do not measure or compare the performance of the two protocols once integrated. Jamal and Abdellah [15] use a mathematical model based on queueing theory to estimate how HTTP, MQTT, and CoAP behave under traffic load in a oneM2M deployment, without validating the estimates against real hardware measurements. Medjhah et al. [14] simulate a oneM2M-based system for remote patient monitoring and evaluate response times and resource usage, but test only a single protocol configuration and do not compare bindings against each other. WebSocket does not appear as a transport option in any of these works, and none address UxV workloads.

The integration of UxVs with IoT communication platforms has relied predominantly on MAVLink, a lightweight binary protocol designed for direct communication between a vehicle and its ground control station [5]. While effective for point-to-point control, MAVLink-based architectures do not provide the resource management, access control, or multi-stakeholder interoperability that a standardized service layer offers. Within this context, da Silva et al. [17] build and test an MQTT-based platform for Unmanned Aerial

Vehicle (UAV) telemetry and command, measuring latency and QoS across three delivery guarantee levels, but without any service layer above the broker. The possibility of using oneM2M as a UAV management platform was explored early by Choi et al. [18], who connected a drone to a oneM2M server through a translation component that bridges the drone’s proprietary protocol with the oneM2M interface. However, that work dates from 2017, covers only HTTP as the underlying transport, and does not evaluate latency or throughput. No prior work has placed a UxV within a oneM2M architecture and empirically compared all four native transport bindings under realistic telemetry and command workloads.

Table 1 positions the reviewed studies across eight dimensions: the four oneM2M transport bindings (HTTP, MQTT, CoAP, and WebSocket), the presence of a oneM2M service layer, whether measurements were conducted on real hardware (Empirical), whether protocol performance was quantitatively compared (Bench.), and whether the platform was validated under UxV workloads. This work is the only one to satisfy all eight criteria simultaneously.

Table 1. Summary of related work. ✓ indicates the property is present; × indicates it is absent. WS = WebSocket; Bench. = Benchmarked.

Study	HTTP	MQTT	CoAP	WS	oneM2M	Empirical	Bench.	UxV
Wytrębowicz et al. [16]	✓	✓	✓	✓	×	×	×	×
Silva et al. [11]	×	✓	✓	×	×	✓	✓	×
Amirkhanov et al. [12]	✓	✓	×	✓	×	✓	✓	×
Et-Tousy and Zyane [13]	×	✓	✓	×	✓	×	×	×
Jamal and Abdellah [15]	✓	✓	✓	×	✓	×	×	×
Medjhah et al. [14]	×	×	×	×	✓	×	×	×
da Silva et al. [17]	×	✓	×	×	×	✓	✓	✓
Choi et al. [18]	✓	×	×	×	✓	✓	×	✓
This work	✓	✓	✓	✓	✓	✓	✓	✓

3. System Architecture

This section describes the architecture of the benchmark system used to evaluate the performance of four oneM2M transport bindings under real UxV workloads.

3.1. Overview

The benchmark system comprises three components connected over a local-area network, shown in Figure 1.

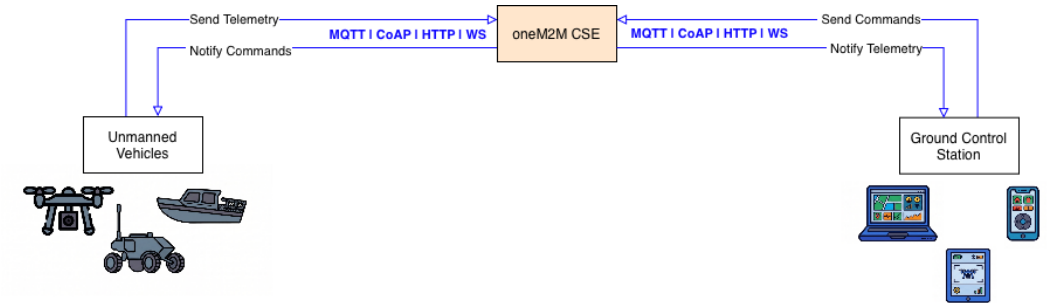


Figure 1. Benchmark system architecture.

The UxV is represented by a platform capable of telemetry reporting and command execution. A oneM2M Application Entity (AE) is registered on the CSE to represent the vehicle in the resource tree, bridging the UxV communication interface with the oneM2M service layer. The **oneM2M CSE** acts as the Infrastructure Node: it stores all resources, routes

subscription notifications, and exposes the four transport bindings. The **Ground Control Station (GCS)** orchestrates benchmark runs, subscribes to telemetry and acknowledgment containers, and dispatches commands. The UxV and GCS communicate exclusively through the CSE using the oneM2M resource model.

3.2. OneM2M Resource Tree

All resources reside under the CSE-Base. The UxV creates the AE and its three child containers on first connection. The GCS creates its subscriptions once the containers exist, as shown in Figure 2.

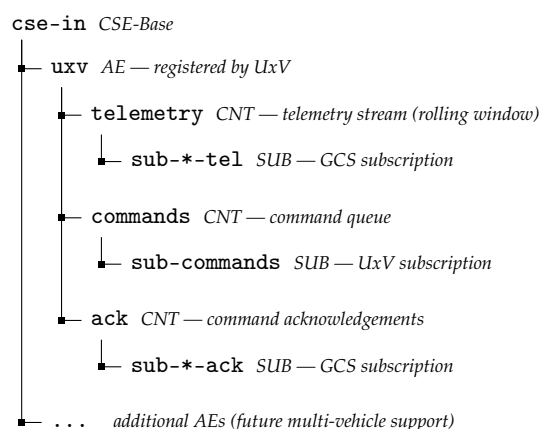


Figure 2. oneM2M resource tree. sub-* expands per protocol binding.

The `telemetry` container receives Content Instances (CINs) from the UxV at regular intervals. Each new CIN is appended to the container and the oldest instance is discarded when the capacity limit is reached, forming a rolling window of the most recent telemetry snapshots. The `commands` container acts as a command queue: the GCS publishes command CINs, the UxV receives them via subscription notification, and dispatches each to the flight controller. The `ack` container closes the measurement loop: after each command is dispatched, the UxV publishes an acknowledgment CIN carrying delivery timestamps, which the GCS uses to compute command delivery latency.

3.3. Per-Protocol Bindings

Each transport binding uses the same oneM2M resource model and message semantics but differs in connection model, notification delivery, and framing overhead. Table 2 summarizes the key characteristics of each binding.

Table 2. oneM2M transport binding characteristics.

Protocol	Transport	Connection model	Notification delivery
WebSocket	TCP	Persistent, bidirectional	CSE reuses active connection
MQTT	TCP, QoS 1	Persistent, pub/sub via broker	CSE publishes to subscriber topic
HTTP	TCP	Stateless, per-request	CSE POSTs to callback server
CoAP	UDP (CON)	Stateless, per-datagram	CSE POSTs to callback server

WebSocket reuses a single persistent TCP connection for both requests and notifications. MQTT routes all messages through a broker, adding one hop but decoupling delivery from the CSE event loop. HTTP opens a new TCP connection per request and delivers

notifications via CSE POST to a callback server. CoAP uses Confirmable UDP datagrams with per-message acknowledgment, providing reliability without a persistent connection.

4. Methodology

This section describes the implementation choices, experimental scenarios, and metrics used to evaluate the four oneM2M transport bindings.

4.1. Implementation & Experimental Environment

The UxV instance was implemented using a DJI Mavic 2 Enterprise Advanced (M2EA), operated via a DJI RC remote controller running Android. A native Android application was developed in Java using DJI Mobile SDK v4. On startup, it registers the uxv AE on the CSE and creates the resource tree. During operation, it publishes drone telemetry as CINs to the telemetry container and subscribes to the commands container to receive and execute commands dispatched by the GCS.



Figure 3. Experimental UxV setup: DJI M2EA drone and DJI RC remote controller.

The GCS was implemented as a Python application using Streamlit as the orchestration interface. It subscribes to the telemetry and ack containers, publishes commands to the commands container, orchestrates benchmark runs, and logs per-message metrics to CSV for offline analysis. The GCS ran on the same machine as the ACME CSE, sharing the local network interface for all protocol communications.

For the oneM2M CSE, ACME CSE v2025.11 was selected and deployed via Docker Compose alongside an Eclipse Mosquitto MQTT broker. The CSE uses TinyDB as its database backend, a file-based JSON store. Two patches were applied to the CSE source: disabling the WebSocket keepalive ping scheduler, which caused the Android client to disconnect under benchmark load, and adding `Operation.NOTIFY` to the CoAP operation map, which was absent in v2025.11 and raised a `KeyError` on every CoAP notification delivery attempt.

A lab constraint affects CoAP notification delivery. Docker Desktop does not route UDP packets originating inside a container to external LAN devices, preventing the CSE from delivering CoAP NOTIFY messages over UDP. Both the UxV and the GCS register HTTP/TCP callback servers as their notification addresses, while all outgoing CoAP requests continue to use CoAP/UDP. This constraint adds TCP connection overhead to CoAP notification latency and is reported as a limitation in Section 6.3.

All benchmark runs were conducted on a laptop running Windows 11, equipped with an AMD Ryzen 5 5500U processor and 40 GB of RAM. The UxV RC and development machine were connected over a single 802.11 WiFi LAN using an Intel Wi-Fi 6 AX200 adapter, with a measured baseline round-trip time of 10 ms on average (5–198 ms range, 0% packet loss, $n=100$) between the machine and the UxV RC.

4.2. Benchmark Scenarios

Three benchmark scenarios were defined to evaluate distinct aspects of UxV communication performance. Scenarios 1 and 2 assess telemetry uplink under different load conditions, while Scenario 3 assesses command delivery. Each scenario runs for 60 seconds and is repeated 10 times per protocol, yielding 120 runs in total.

4.2.1. Scenario 1 — Telemetry Uplink at 4 msg/s

The UxV publishes telemetry CINs to the telemetry container at 4 messages per second (250 ms interval), representing a single-drone operational load. Each run lasts 60 seconds, yielding 240 expected messages per run. The GCS subscribes to the telemetry container and records a `timestamp_ms` upon each notification receipt. The UxV embeds a `t_send_ms` timestamp in each CIN. As both timestamps originate from different devices, the derived latency values are subject to inter-device clock offset and are excluded from inter-protocol comparison.

4.2.2. Scenario 2 — Telemetry Uplink at 16 msg/s

Identical to Scenario 1 but at 16 messages per second (62 ms interval), simulating the aggregate telemetry load of four simultaneous drones. Each run yields 960 expected messages. This rate was selected to stress-test the CSE notification pipeline and evaluate protocol behavior under sustained high load.



Figure 4. Scenarios 1 and 2 data flow. Latency is not used for inter-protocol comparison due to unsynchronized device clocks.

4.2.3. Scenario 3 — Command Round-Trip

The GCS dispatches 60 commands to the commands container at approximately one command per second, with a 15-second acknowledgment timeout per command. The UxV receives each command via subscription notification and immediately publishes an acknowledgment CIN to the ack container, without invoking the UxV SDK. This isolates pure protocol latency from flight controller execution time.

Two latency measurements are captured per command. The primary metric, `cin_create_ms`, is the round-trip time between command CIN publication and CSE acknowledgment (`rsc=2001`), measured entirely on the GCS using a monotonic clock with no inter-device clock dependency. A secondary metric, `latency_ms` (`t_recv_ms - t_cmd_ms`), captures the time between command dispatch at the GCS and receipt at the UxV, but is subject to inter-device clock offset and is reported for reference only.

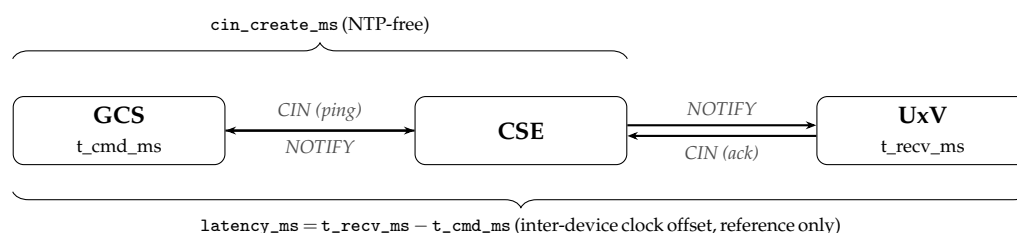


Figure 5. Scenario 3 data flow. `cin_create_ms` is the primary latency metric (GCS-to-CSE round-trip, same device). `latency_ms` captures GCS-to-UxV one-way time but is subject to inter-device clock offset.

4.3. Metrics

Five metrics are collected per message across the scenarios, as defined in Table 3.

Table 3. Performance metrics collected per message.

Metric	Definition
Latency (ms)	S3: <code>cin_create_ms</code> — GCS-to-CSE round-trip on a monotonic clock (NTP-free, primary metric). S1/S2: <code>latency_ms</code> = <code>timestamp_ms</code> − <code>t_send_ms</code> , reported as approximation only; subject to inter-device clock offset (≈ 1761 – 2562 ms) and excluded from inter-protocol comparison.
Throughput (msg/s)	Delivered messages divided by observed elapsed time, from first to last received message within the run.
Packet loss (%)	S1/S2: gaps in the monotonic seq counter, $1 - n_{\text{received}} / (\text{seq}_{\text{max}} - \text{seq}_{\text{min}} + 1)$. S3: fraction of commands without ACK within the 15 s timeout.
Protocol overhead (%)	$\text{header_bytes} / (\text{header_bytes} + \text{payload_bytes}) \times 100$, computed per message.
Jitter (ms)	S1/S2: standard deviation of <code>latency_ms</code> within a run. S3: standard deviation of <code>cin_create_ms</code> within a run.

Latency values for S1 and S2 are subject to inter-device clock offset, as the UxV and GCS clocks were not synchronized. For each S3 run, the per-run offset was estimated as `cin_create_ms`/2 − `latency_ms`. The estimated drift totals approximately 800 ms over the 9-hour session. Corrected S1 values are provided for reference but excluded from inter-protocol comparison due to estimation uncertainty.

Protocol overhead is computed per message using protocol-specific methods. For WebSocket, the frame header is computed analytically from payload size per RFC 6455 (6 bytes for payloads ≤ 125 bytes, 8 bytes for ≤ 65535 bytes, 14 bytes otherwise) [19]. For MQTT, header overhead includes the fixed header, the variable-length remaining-length field, and the topic string, computed per message following the MQTT v3.1.1 specification [20]. For HTTP, request and response header bytes are measured directly from each exchange. For CoAP, the notification header bytes are measured from the first incoming HTTP callback (lab constraint — CoAP notifications are delivered via HTTP/TCP).

5. Results

This section presents the benchmark results across the three scenarios and four protocols. All values represent means over ten independent runs unless stated otherwise. Latency in Scenarios 1 and 2 is subject to inter-device clock offset (Section 4) and approximate corrected values are included for reference but excluded from inter-protocol comparison. Scenario 3 `cin_create_ms` is the sole NTP-independent latency metric and is used for all latency comparisons. Table 4 provides a consolidated overview of all metrics. MQTT demonstrates the most balanced profile, combining low latency, zero packet loss in S1 and S2, and moderate overhead. CoAP achieves the lowest command round-trip latency in S3 and minimal jitter, but suffers from catastrophic packet loss (69.9%) under high load in S2 due to UDP's lack of flow control. WebSocket imposes near-zero framing overhead and handles S2 load gracefully, but exhibits significant command loss (21.2%) in S3 because WebSocket notifications share the CSE asyncio event loop with TinyDB writes, which periodically stall delivery past the acknowledgment timeout. HTTP presents the highest latency and overhead, yet maintains reliable delivery within the CSE processing ceiling. The subsections that follow analyze each scenario in detail.

Table 4. Performance summary per protocol and scenario ($n = 10$ runs). S1 latency is NTP-corrected approximation ($\dagger$). Excluded from inter-protocol comparison. S2 latency omitted (TinyDB backlog contamination). Overhead computed from S1 runs. $\ddagger$ CoAP overhead measured via HTTP/TCP callback (lab constraint) does not represent intrinsic CoAP/UDP framing cost.

Scenario	Metric	MQTT	CoAP	WebSocket	HTTP
S1 — 4 msg/s	Latency †	100 ms	109 ms	133 ms	182 ms
	Packet loss	0.0%	1.5%	0.0%	1.3%
	Jitter	15.7 ms	25.4 ms	25.5 ms	150.1 ms
	Throughput	4.05 msg/s	3.96 msg/s	4.04 msg/s	4.00 msg/s
S2 — 16 msg/s	Packet loss	0.0%	69.9%	0.7%	0.3%
	Throughput	4.63 msg/s	6.49 msg/s	5.19 msg/s	3.87 msg/s
S3 — Command	cin_create	102.6 ms	80.4 ms	144.1 ms	105.9 ms
	Jitter	16.7 ms	8.2 ms	14.2 ms	17.4 ms
	Packet loss	0.0%	0.2%	21.2%	0.3%
Overhead	(S1 runs)	5.7%	35.6% $\ddagger$	1.2%	47.6%

5.1. Scenario 1 — Telemetry Uplink at 4 msg/s

All four protocols sustain the 4 msg/s target throughput under low load, with effective delivery rates between 3.96 and 4.05 msg/s (Table 4), as illustrated in Figure 6. MQTT and WebSocket achieve zero packet loss across all ten runs. CoAP exhibits 1.5% mean loss, attributable to occasional UDP drops under the CSE async scheduler. HTTP shows 1.3% loss from connection timeout events.

Approximate NTP-corrected S1 latency values are shown in Figure 7. The inter-device clock offset was estimated separately for each protocol from S3 anchor runs collected at different times. The estimated offsets range from approximately 1761 ms (WebSocket) to 2562 ms (CoAP), with a total drift of approximately 800 ms over the 9-hour collection session. Because each protocol's correction uses a different anchor and a different extrapolation interval, the resulting values carry protocol-specific estimation uncertainty and cannot be directly ranked against each other. They are included as reference only: MQTT $\approx$ 100 ms, CoAP $\approx$ 109 ms, WebSocket $\approx$ 133 ms, HTTP $\approx$ 182 ms.

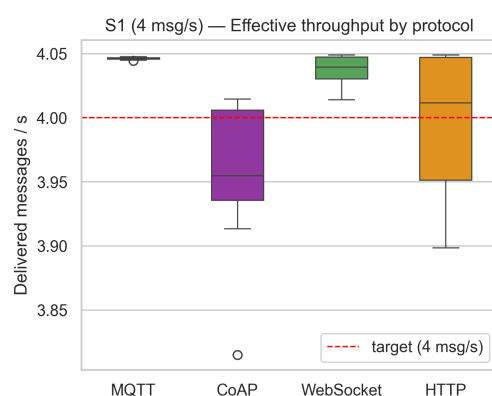


Figure 6. Effective throughput in S1 (4 msg/s target, $n = 10$ runs per protocol).

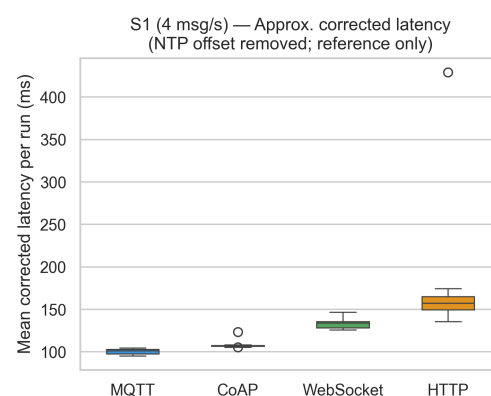


Figure 7. Approximate NTP-corrected S1 latency per protocol ($n = 10$ runs). Values are reference only (Section 4).

Protocol overhead at 4 msg/s is protocol-intrinsic for WebSocket (1.2%) and MQTT (5.7%), which impose minimal framing. HTTP (47.6%) reflects verbose HTTP/1.1 headers per exchange. CoAP (35.6%) reflects the HTTP/TCP callback delivery workaround rather than intrinsic CoAP/UDP framing cost (Section 5.4).

5.2. Scenario 2 — Telemetry Uplink at 16 msg/s

At 16 msg/s, no protocol reaches the nominal target throughput. ACME CSE v2025.11 processes subscription notifications synchronously inside its asyncio event loop; TinyDB persistence writes block the loop and cap effective notification delivery at approximately 4–5 msg/s, regardless of which transport binding is used. This ceiling dominates S2 results.

As shown in Figure 8, CoAP exhibits catastrophic packet loss (69.9%). At 16 msg/s the CSE asyncio queue fills faster than it drains. Because CoAP uses UDP, datagrams that arrive while the queue is saturated are silently dropped at the network layer with no retransmission. MQTT (0.0% loss, 4.63 msg/s) and WebSocket (0.7% loss, 5.19 msg/s) use TCP, so messages that arrive above the ceiling are buffered in the TCP stack and delivered after the run window rather than dropped. They achieve lower measured packet loss at the cost of growing end-to-end latency. HTTP (0.3% loss, 3.87 msg/s) stays within the ceiling and avoids backlog, but achieves the lowest effective throughput of the four protocols.

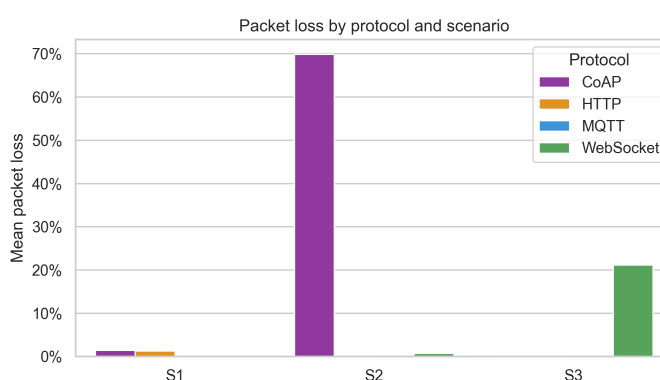


Figure 8. Mean packet loss per protocol and scenario ($n = 10$ runs). CoAP exhibits catastrophic loss in S2 (69.9%) due to CSE asyncio saturation and UDP drops. WebSocket S3 loss (21.2%) results from TinyDB writes stalling the CSE asyncio loop past the GCS acknowledgment timeout.

Reported S2 throughput is computed over the window from first to last received message, which understates true load under heavy loss. CoAP delivers an early burst before the queue saturates and then drops most subsequent datagrams, so its 6.49 msg/s reflects a short high-rate window rather than sustained delivery above the CSE ceiling.

S2 latency values are not reported: the TinyDB notification backlog introduces stale-notification contamination between runs, inflating measured latency by tens of seconds and rendering the values unrepresentative of true delivery time.

5.3. Scenario 3 — Command Round-Trip Latency

Figure 9 and Table 4 summarize `cin_create_ms` results for S3. CoAP achieves the lowest command round-trip latency (80.4 ± 1.4 ms), followed by MQTT (102.6 ± 5.1 ms) and HTTP (105.9 ± 3.9 ms). WebSocket records the highest latency (144.1 ± 1.7 ms).

WebSocket also exhibits 21.2% command loss in S3, compared to near-zero loss for CoAP (0.2%), HTTP (0.3%), and MQTT (0.0%), as visible in Figure 8. This loss has the same root cause as the S2 ceiling: ACME CSE v2025.11 delivers WebSocket notifications directly through its asyncio event loop, which competes with TinyDB writes. Under command load, TinyDB operations periodically stall the loop long enough for individual notification deliveries to exceed the 15 s acknowledgment timeout imposed by the GCS, causing those commands to be recorded as lost. MQTT, HTTP, and CoAP are not affected because their notification paths do not share the asyncio loop: MQTT delegates delivery to the Mosquitto broker, and HTTP and CoAP use separate callback servers on the GCS side.

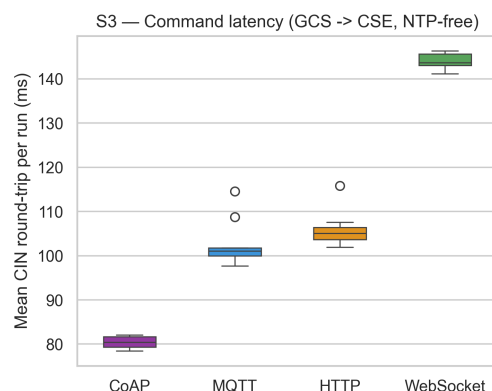


Figure 9. `cin_create_ms` per protocol in S3 ($n = 10$ runs). CoAP achieves the lowest median latency (78 ms); WebSocket the highest (141 ms).

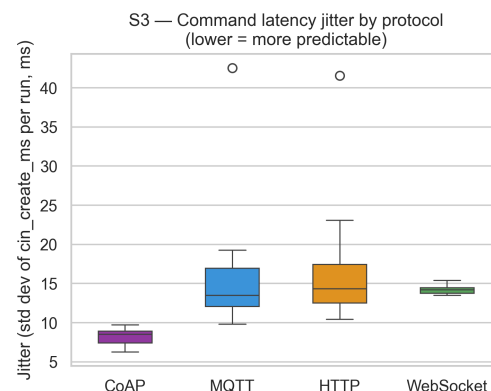


Figure 10. Jitter of `cin_create_ms` within a run for S3 ($n = 10$ runs). CoAP exhibits the lowest jitter (8.2 ms).

As shown in Figure 10, CoAP achieves the lowest jitter (8.2 ms), confirming consistent delivery timing; MQTT (16.7 ms), HTTP (17.4 ms), and WebSocket (14.2 ms) exhibit comparable, moderate jitter.

5.4. Protocol Overhead

Protocol overhead is consistent across scenarios and reflects framing characteristics independent of load, as depicted in Figure 11. WebSocket (1.2%) imposes near-zero per-message overhead owing to its persistent connection and minimal frame header. MQTT (5.7%) adds a compact fixed header plus topic string. HTTP overhead (47.6%–90.7%) reflects the verbosity of HTTP/1.1 request and response headers per message exchange.

CoAP overhead requires a specific note. In S1 (35.6%), the value is measured from the incoming HTTP callback headers, because the lab constraint routes CSE-to-GCS CoAP notifications via HTTP/TCP (Section 4). In S3 (85.5%), the overhead rises further because S3 command CINs are smaller payloads, making the fixed HTTP callback header a larger fraction of the total bytes. Neither value represents the intrinsic framing overhead of CoAP/UDP, which has a 4-byte fixed header and is substantially lower. CoAP overhead figures should be interpreted as the overhead of the notification delivery workaround used in this lab environment, not as a property of CoAP itself.

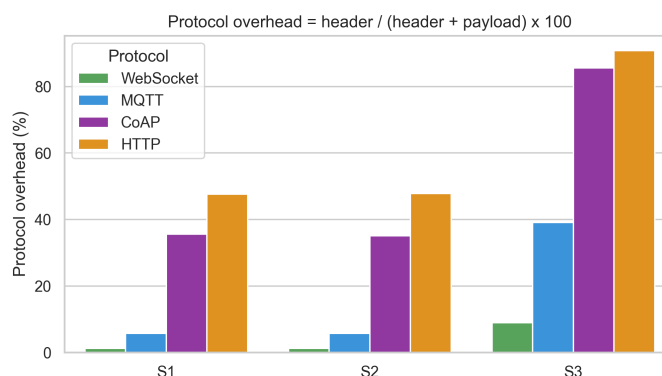


Figure 11. Protocol overhead (%) per scenario. WebSocket and MQTT impose minimal framing overhead.

6. Discussion

6.1. Protocol Trade-offs

Based on the results of this evaluation, oneM2M as implemented in ACME CSE v2025.11 with TinyDB is not viable for real-time UxV operations. The notification pipeline caps effective delivery at approximately 4–5 msg/s — a ceiling that the aggregate load of as few as four simultaneous drones saturates entirely. This ceiling is a property of the synchronous TinyDB persistence layer, not of the oneM2M standard or its transport bindings, whose architectural properties remain well-suited to unmanned systems.

Within this constraint, the key differentiator across protocols is not raw speed but how each binding interacts with the CSE notification pipeline. MQTT delivers the most balanced profile: zero packet loss at both telemetry rates, the lowest telemetry jitter (15.7 ms), and competitive command latency (102.6 ms). The Mosquitto broker adds one network hop but isolates notification delivery from CSE internal scheduling, which proves decisive at high load. WebSocket achieves near-zero framing overhead (1.2%) and zero packet loss at 4 msg/s, but its notifications share the CSE asyncio event loop with TinyDB writes, producing 21.2% command loss in S3 and the highest round-trip latency (144.1 ms). CoAP achieves the lowest command latency (80.4 ms) and jitter (8.2 ms), but UDP provides no flow control: at 16 msg/s, 69.9% of datagrams are silently dropped when the CSE queue saturates. HTTP is the most reliable binding under load (0.3% loss at 16 msg/s) but the least efficient, with the highest framing overhead (47.6%) and the highest command round-trip latency among the reliable bindings (105.9 ms).

6.2. Implications for UxV Operations

Real-world UxV missions — search and rescue, infrastructure inspection, precision agriculture — typically operate fleets of two or more vehicles simultaneously. At 4 msg/s per vehicle, a fleet of four drones generates 16 msg/s of aggregate telemetry, saturating the CSE ceiling entirely. Under these conditions, the choice of transport binding determines the failure mode rather than performance: TCP-based protocols buffer excess messages with growing latency, while CoAP drops them silently. No binding delivers acceptable performance above the ceiling.

Replacing TinyDB with a production-grade backend such as PostgreSQL is therefore a necessary precondition for oneM2M to be viable in practice. For single-vehicle deployments within the tested rates, MQTT is the recommended binding for its delivery isolation and zero packet loss. For command dispatch, MQTT and HTTP offer the best balance of latency and reliability. CoAP achieves the lowest round-trip time but is not recommended under concurrent telemetry load. WebSocket is not recommended for command dispatch due to the 21.2% command loss caused by CSE asyncio coupling.

6.3. Limitations

Several limitations should be considered when interpreting the results of this study.

Approximate telemetry latency. In Scenarios 1 and 2, latency is derived from timestamps recorded on two different devices without clock synchronization. The inter-device clock offset was estimated at approximately 1761–2562 ms across protocols, reflecting a drift of 801 ms over the session. Latency values are excluded from inter-protocol comparison.

CoAP notification constraint. Docker Desktop does not route UDP packets from containers to external LAN devices. CoAP notifications are therefore delivered over HTTP/TCP, inflating CoAP notification latency relative to a native UDP deployment. Outgoing CoAP requests are unaffected.

CSE throughput ceiling. ACME CSE v2025.11 with TinyDB sustains a maximum notification delivery rate of approximately 4–5 msg/s. Results at 16 msg/s reflect the behavior of the CSE implementation rather than the transport binding alone.

Protocol overhead estimation. WebSocket and MQTT overhead is computed analytically. CoAP overhead is measured from HTTP/TCP callback headers due to the lab constraint and does not represent intrinsic CoAP/UDP framing cost.

Controlled lab environment. All runs were conducted on a single indoor WiFi LAN with no mobility, handover, or artificial congestion. Results may not generalize to outdoor deployments with long-range radio links, 4G/5G connectivity, or degraded network conditions.

Single UxV platform. The benchmark uses a single DJI M2EA drone. Generalization to other UxV platforms or SDKs requires further evaluation.

7. Conclusions

This paper presented the first empirical benchmark of all four oneM2M transport bindings — WebSocket, MQTT, HTTP, and CoAP — on a live UxV platform, addressing the question of whether oneM2M is a viable service layer for real-time UxV operations and which binding best supports telemetry and command workloads.

A benchmark system comprising a DJI Mavic 2 Enterprise Advanced drone, an ACME CSE v2025.11 instance, and a Python-based GCS was evaluated across three operational scenarios and 120 runs. The central finding is that oneM2M as implemented in ACME CSE v2025.11 with TinyDB is not viable for real-time UxV operations. The notification pipeline caps effective delivery at approximately 4–5 msg/s — a ceiling that the aggregate load of as few as four simultaneous drones saturates entirely. This is a CSE implementation constraint, not a limitation of the oneM2M standard, whose architecture remains well-suited to unmanned systems.

Within the tested constraints, MQTT is the recommended binding for deployments combining telemetry and command workloads: zero packet loss at both telemetry rates, the lowest telemetry jitter (15.7 ms), and competitive command latency (102.6 ms). WebSocket is preferable when framing overhead is the dominant constraint (1.2%). CoAP achieves the lowest command round-trip time (80.4 ms) but is not suitable under concurrent telemetry load due to UDP's lack of flow control. HTTP provides the highest reliability under load (0.3% packet loss at 16 msg/s) but carries the highest framing overhead (47.6%).

Supplementary Materials: The benchmark system, including all source code, configuration files, raw data, and analysis scripts, is available at <https://github.com/j-parreira/OneM2M-UxV-Project>.

Use of Artificial Intelligence: During the preparation of this manuscript and the development of the benchmark software, the authors used large language model (LLM) assistants for language editing, writing assistance, and code development. The authors have reviewed and edited all outputs and take full responsibility for the content of this publication and the correctness of the software.

References

- Adil, M.; Song, H.; Jan, M.A.; Khan, M.K.; He, X.; Farouk, A.; Jin, Z. UAV-Assisted IoT Applications, QoS Requirements and Challenges with Future Research Directions. *56*, 251:1–251:35. <https://doi.org/10.1145/3657287>.
- Bae, I.; Hong, J. Survey on the Developments of Unmanned Marine Vehicles: Intelligence and Cooperation. *23*, 4643. <https://doi.org/10.3390/s23104643>.
- Munasinghe, I.; Perera, A.; Deo, R.C. A Comprehensive Review of UAV-UGV Collaboration: Advancements and Challenges. *13*, 81. <https://doi.org/10.3390/jsan13060081>.
- Wibisono, A.; Piran, M.J.; Song, H.K.; Lee, B.M. A Survey on Unmanned Underwater Vehicles: Challenges, Enabling Technologies, and Future Research Directions. *23*, 7321. <https://doi.org/10.3390/s23177321>.

5. Ramos, J.; Ribeiro, R.; Safadinho, D.; Barroso, J.; Rabadão, C.; Pereira, A. Distributed Architecture for Unmanned Vehicle Services. *21*, 1477. <https://doi.org/10.3390/s21041477>. 390
6. oneM2M Functional Architecture (TS-0001, V4.22.0). Available online: <https://member.onem2m.org/Application/documentapp/downloadLatestRevision/default.aspx?docID=38119> (accessed on 2026-06-02). 391
7. oneM2M HTTP Protocol Binding (TS-0009, V4.6.1). Available online: <https://member.onem2m.org/Application/documentapp/downloadLatestRevision/default.aspx?docID=38217> (accessed on 2026-06-02). 392
8. oneM2M WebSocket Protocol Binding (TS-0020, V4.0.0). Available online: <https://member.onem2m.org/Application/documentapp/downloadLatestRevision/default.aspx?docID=35387> (accessed on 2026-06-02). 393
9. oneM2M MQTT Protocol Binding (TS-0010, V4.1.0). Available online: <https://member.onem2m.org/Application/documentapp/downloadLatestRevision/default.aspx?docID=38215> (accessed on 2026-06-02). 394
10. oneM2M CoAP Protocol Binding (TS-0008, V4.6.1). Available online: <https://member.onem2m.org/Application/documentapp/downloadLatestRevision/default.aspx?docID=37681> (accessed on 2026-06-02). 395
11. Silva, D.; Carvalho, L.I.; Soares, J.; Sofia, R.C. A Performance Analysis of Internet of Things Networking Protocols: Evaluating MQTT, CoAP, OPC UA. *11*, 4879. <https://doi.org/10.3390/app11114879>. 396
12. Amirkhanov, B.; Amirkhanova, G.; Kunelbayev, M.; Adilzhanova, S.; Tokhtassyn, M. Evaluating HTTP, MQTT over TCP and MQTT over WEBSOCKET for Digital Twin Applications: A Comparative Analysis on Latency, Stability, and Integration. *International Journal of Innovative Research and Scientific Studies* **2025**, *8*, 679–694. <https://doi.org/10.53894/ijirss.v8i1.4414>. 397
13. Et-Tousy, J.; Zyane, A. Integration of MQTT and CoAP Protocols in OneM2M for IoT Applications. In Proceedings of the Intersection of Artificial Intelligence, Data Science, and Cutting-Edge Technologies: From Concepts to Applications in Smart Environment; Farhaoui, Y.; Herawan, T.; Lucky Imoize, A.; Allaoui, A.E., Eds. Springer Nature Switzerland, pp. 185–195. https://doi.org/10.1007/978-3-031-90921-4_26. 398
14. Medjeh, S.; Monteil, T.; Peraldi-Frati, M.A.; Liquori, L. Multi-Layered Model for Performance Evaluation of oneM2M-Based IoT Solution. In Proceedings of the Internet of Things; Rey, G.; Tigli, J.Y.; Franquet, E., Eds. Springer Nature Switzerland, pp. 77–95. https://doi.org/10.1007/978-3-031-81900-1_5. 399
15. Jamal, E.T.; Abdellah, Z. Comprehensive Analysis, Modeling, and Optimization of QoS in OneM2M for IoT Applications: Integrating HTTP, MQTT, and CoAP Protocols With Automatic Traffic SLA Management. *38*, e70227. <https://doi.org/10.1002/dac.70227>. 400
16. Wyrębowicz, J.; Cabaj, K.; Krawiec, J. Messaging Protocols for IoT Systems—A Pragmatic Comparison. *21*, 6904. <https://doi.org/10.3390/s21206904>. 401
17. da Silva, L.M.; Menezes, H.B.d.B.; Luccas, M.d.S.; Mailer, C.; Pinto, A.S.R.; Boava, A.; Rodrigues, M.; Ferrão, I.G.; Estrella, J.C.; Branco, K.R.L.J.C. Development of an Efficiency Platform Based on MQTT for UAV Controlling and DoS Attack Detection. *Sensors* **2022**, *22*, 6567. <https://doi.org/10.3390/s22176567>. 402
18. Choi, S.C.; Sung, N.M.; Park, J.H.; Ahn, I.Y.; Kim, J. Enabling Drone as a Service: OneM2M-based UAV/Drone Management System. In Proceedings of the 2017 Ninth International Conference on Ubiquitous and Future Networks (ICUFN), pp. 18–20. <https://doi.org/10.1109/ICUFN.2017.7993739>. 403
19. The WebSocket Protocol. Available online: <https://datatracker.ietf.org/doc/rfc6455> (accessed on 2026-06-02). 404
20. MQTT Version 5.0 Specification. Available online: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.pdf> (accessed on 2026-06-05). 405